CS2023 Algoritmos y Estructuras de Datos
Problemas con recursión y backtracking
Semana 10

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

13 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Una receta para diseñar soluciones recursivas

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Una receta para diseñar soluciones recursivas
- Recursión que sigue la estructura del problema: Torres de Hanói y expresiones en notación prefija

Objetivos de la sesión

- Una receta para diseñar soluciones recursivas
- Recursión que sigue la estructura del problema: Torres de Hanói y expresiones en notación prefija
- Backtracking con buenas reglas de poda: paréntesis balanceados, permutaciones con letras repetidas y Sudoku

Una receta para pensar en recursivo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1. El predicado

Qué calcula la función, dicho en palabras, y con qué argumentos. Es el origen de la recursión: la pregunta es qué respuesta de un problema del mismo tipo, pero más chico, ayuda a armar la actual.

Una receta para pensar en recursivo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1. El predicado

Qué calcula la función, dicho en palabras, y con qué argumentos. Es el origen de la recursión: la pregunta es qué respuesta de un problema del mismo tipo, pero más chico, ayuda a armar la actual.

2. Los casos base

Los argumentos que se resuelven directo, sin llamarse a sí misma.

Una receta para pensar en recursivo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1. El predicado

Qué calcula la función, dicho en palabras, y con qué argumentos. Es el origen de la recursión: la pregunta es qué respuesta de un problema del mismo tipo, pero más chico, ayuda a armar la actual.

2. Los casos base

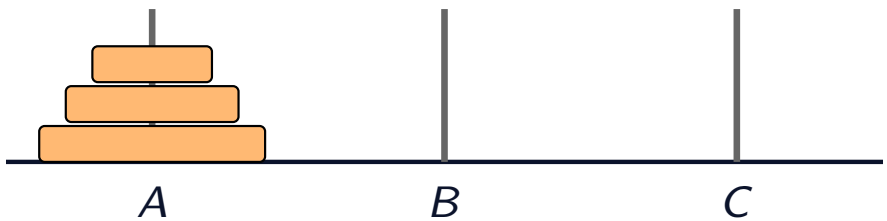
Los argumentos que se resuelven directo, sin llamarse a sí misma.

3. El paso recursivo: confiar en el predicado

Se arma la respuesta suponiendo que las llamadas con argumentos más chicos cumplen el predicado. No hace falta seguirlas hasta el fondo; eso solo sirve para estimar el costo. Si el paso no sale, muchas veces el problema es el predicado.

Torres de Hanói

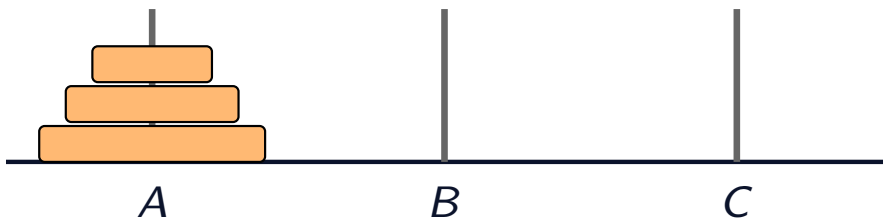
Torres de Hanói



El problema

Hay n discos de tamaños distintos en la varilla A, del más grande (abajo) al más chico. Hay que pasarlos todos a la varilla C, moviendo un disco a la vez y sin poner nunca un disco sobre uno más chico. La varilla B sirve de apoyo.

Torres de Hanói



El problema

Hay n discos de tamaños distintos en la varilla A, del más grande (abajo) al más chico. Hay que pasarlos todos a la varilla C, moviendo un disco a la vez y sin poner nunca un disco sobre uno más chico. La varilla B sirve de apoyo.

Queremos la lista de movimientos, y saber cuántos hacen falta.

Un primer predicado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primer intento

$f(n)$: el mínimo de movimientos para llevar una torre de n discos de A a C.
Caso base: $f(1) = 1$.

Un primer predicado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primer intento

$f(n)$: el mínimo de movimientos para llevar una torre de n discos de A a C.
Caso base: $f(1) = 1$.

¿Se puede escribir $f(n)$ usando $f(n-1)$?

Un primer predicado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primer intento

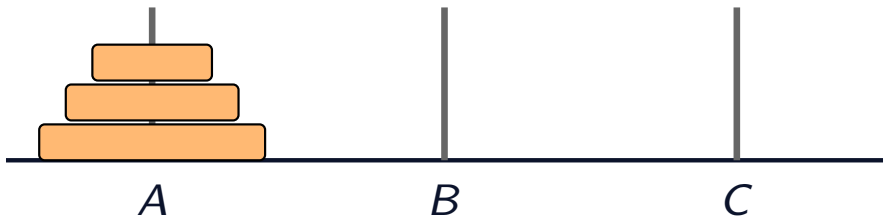
$f(n)$: el mínimo de movimientos para llevar una torre de n discos de A a C.
Caso base: $f(1) = 1$.

¿Se puede escribir $f(n)$ usando $f(n-1)$?

No con este predicado: es demasiado estrecho

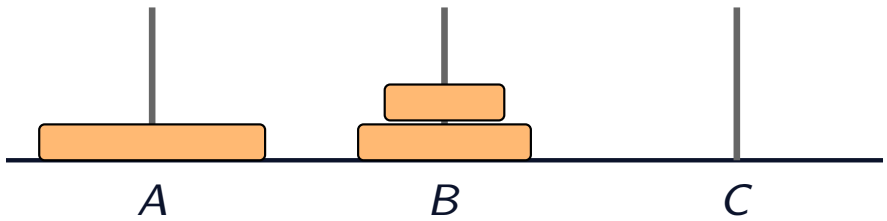
Para mover el disco más grande a C, los $n-1$ de arriba tienen que estar en B, y f solo sabe llevar discos a C. Se **fortalece** el predicado dándole más libertad: $f(n)$ es el mínimo de movimientos para llevar una torre de n discos de una varilla a **cualquier otra**.

¿Qué instancia más chica ayuda?



Cuando el disco más grande se mueve de A a C, ¿dónde tienen que estar los otros $n - 1$?

¿Qué instancia más chica ayuda?

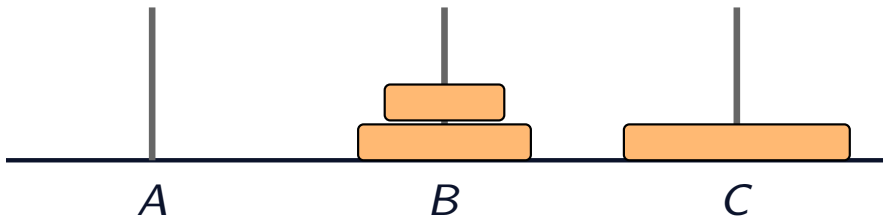


Cuando el disco más grande se mueve de A a C, ¿dónde tienen que estar los otros $n-1$?

Todos en B: es el único lugar libre

Entonces: pasar $n-1$ discos de A a B (el mismo problema, más chico),

¿Qué instancia más chica ayuda?

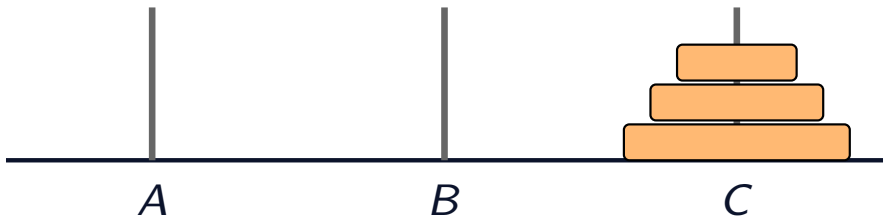


Cuando el disco más grande se mueve de A a C, ¿dónde tienen que estar los otros $n-1$?

Todos en B: es el único lugar libre

Entonces: pasar $n-1$ discos de A a B (el mismo problema, más chico), mover el disco n de A a C,

¿Qué instancia más chica ayuda?



Cuando el disco más grande se mueve de A a C, ¿dónde tienen que estar los otros $n-1$?

Todos en B: es el único lugar libre

Entonces: pasar $n-1$ discos de A a B (el mismo problema, más chico), mover el disco n de A a C, y pasar los $n-1$ de B a C (otra vez el mismo problema).

Torres de Hanói: pseudocódigo

Algoritmo 1: HANOI(n, *origen*, *destino*, *apoyo*)

si $n = 0$ **entonces devolver**
HANOI($n - 1$, *origen*, *apoyo*, *destino*)
imprimir "mover el disco n de *origen* a *destino*"
HANOI($n - 1$, *apoyo*, *destino*, *origen*)

Torres de Hanói: pseudocódigo

Algoritmo 2: HANOI(n, *origen*, *destino*, *apoyo*)

si $n = 0$ **entonces devolver**
HANOI($n-1$, *origen*, *apoyo*, *destino*)
imprimir "mover el disco n de *origen* a *destino*"
HANOI($n-1$, *apoyo*, *destino*, *origen*)

Por qué es correcto

Mientras se mueven los $n-1$ discos de arriba, el disco n está abajo de todo y es más grande que todos: nunca estorba. Basta confiar en que las dos llamadas mueven bien sus $n-1$ discos.

Torres de Hanói: pseudocódigo

Algoritmo 3: HANOI(n, *origen*, *destino*, *apoyo*)

si $n = 0$ **entonces devolver**
HANOI($n-1$, *origen*, *apoyo*, *destino*)
imprimir "mover el disco n de *origen* a *destino*"
HANOI($n-1$, *apoyo*, *destino*, *origen*)

Por qué es correcto

Mientras se mueven los $n-1$ discos de arriba, el disco n está abajo de todo y es más grande que todos: nunca estorba. Basta confiar en que las dos llamadas mueven bien sus $n-1$ discos.

Se llama con HANOI(n, A, C, B). Que *origen* y *destino* sean argumentos **es** el predicado fortalecido: la función sabe mover n discos entre cualquier par de varillas, y en cada llamada cambian de papel.

Torres de Hanói con $n = 3$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1. disco 1: $A \rightarrow C$
2. disco 2: $A \rightarrow B$
3. disco 1: $C \rightarrow B$

Torres de Hanói con $n = 3$

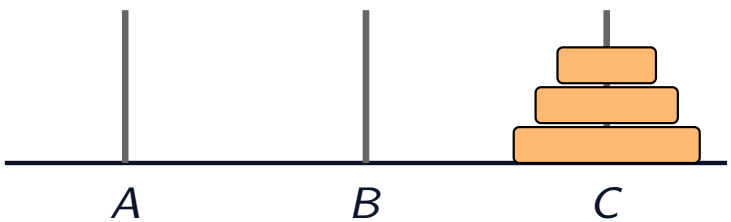
UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1. disco 1: $A \rightarrow C$
2. disco 2: $A \rightarrow B$
3. disco 1: $C \rightarrow B$
4. disco 3: $A \rightarrow C$

; el 4 mueve el disco grande

Torres de Hanói con $n = 3$

1. disco 1: $A \to C$
2. disco 2: $A \to B$
3. disco 1: $C \to B$
4. disco 3: $A \to C$
5. disco 1: $B \to A$
6. disco 2: $B \to C$
7. disco 1: $A \to C$



Los movimientos 1–3 son $\textsc{Hanoi}(2, A, B, C)$; el 4 mueve el disco grande; los 5–7 son $\textsc{Hanoi}(2, B, C, A)$.

¿Cuántos movimientos?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Se cuenta con la misma recursión

$T(0) = 0$ y $T(n) = 2\,T(n-1) + 1$: dos subproblemas de tamaño $n-1$ y un movimiento. Desarrollando: $T(n) = 2^n - 1$ $(1, 3, 7, 15, \ldots)$.

¿Cuántos movimientos?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Se cuenta con la misma recursión

$T(0) = 0$ y $T(n) = 2\,T(n-1) + 1$: dos subproblemas de tamaño $n-1$ y un movimiento. Desarrollando: $T(n) = 2^n - 1$ $(1, 3, 7, 15, \ldots)$.

¿Habrá una forma de hacerlo con menos movimientos?

¿Cuántos movimientos?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Se cuenta con la misma recursión

$T(0) = 0$ y $T(n) = 2\,T(n-1) + 1$: dos subproblemas de tamaño $n-1$ y un movimiento. Desarrollando: $T(n) = 2^n - 1$ (1, 3, 7, 15, ...).

¿Habrá una forma de hacerlo con menos movimientos?

No: $2^n - 1$ es el mínimo

El disco n se mueve al menos una vez, y cuando se mueve por primera vez los otros $n-1$ tienen que estar juntos en la tercera varilla; después hay que volver a juntarlos encima de él. Eso son al menos $T(n-1) + 1 + T(n-1)$ movimientos. Con $n = 64$: unos $1{,}8 \cdot 10^{19}$.

Expresiones en notación prefija

Notación prefija

El operador va antes de sus dos operandos

$* + 2\ 3 - 7\ 4$ es $(2+3)\cdot(7-4) = 15$. No hacen falta paréntesis ni reglas de precedencia.

Notación prefija

El operador va antes de sus dos operandos

$* + 2\ 3 - 7\ 4$ es $(2+3)\cdot(7-4) = 15$. No hacen falta paréntesis ni reglas de precedencia.

Una definición recursiva

Una expresión es un número, o un operador seguido de dos expresiones.

Notación prefija

El operador va antes de sus dos operandos

$* + 2\ 3 - 7\ 4$ es $(2+3)\cdot(7-4) = 15$. No hacen falta paréntesis ni reglas de precedencia.

Una definición recursiva

Una expresión es un número, o un operador seguido de dos expresiones.

Después de $*$, ¿dónde termina el primer operando y empieza el segundo?

Notación prefija

El operador va antes de sus dos operandos

$* + 2\ 3 - 7\ 4$ es $(2+3)\cdot(7-4) = 15$. No hacen falta paréntesis ni reglas de precedencia.

Una definición recursiva

Una expresión es un número, o un operador seguido de dos expresiones.

Después de $*$, ¿dónde termina el primer operando y empieza el segundo?

No hace falta saberlo de antemano

Se evalúa el primer operando desde la posición siguiente; esa misma llamada, al terminar, sabe hasta dónde llegó. La función devuelve dos cosas: el valor y la posición siguiente.

Evaluar: pseudocódigo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 4: EVALUAR(i) — evalúa la expresión que empieza en t_i

si t_i *es un número* **entonces devolver** $(t_i,\ i+1)$
$(x, j) \leftarrow$ EVALUAR$(i+1)$ `// primer operando`
$(y, k) \leftarrow$ EVALUAR(j) `// segundo operando`
devolver $(x\ t_i\ y,\ k)$ `// aplicar el operador` t_i

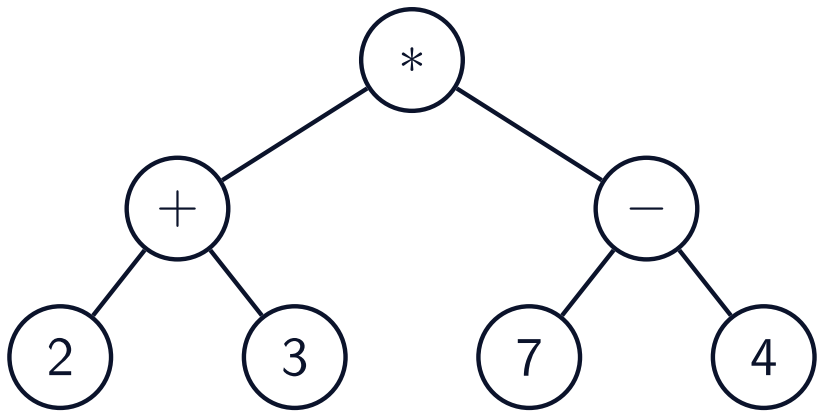
Evaluar: pseudocódigo

Algoritmo 5: EVALUAR(i) — evalúa la expresión que empieza en t_i

si t_i *es un número* **entonces devolver** $(t_i,\ i+1)$
$(x, j) \leftarrow$ EVALUAR$(i+1)$ `// primer operando`
$(y, k) \leftarrow$ EVALUAR(j) `// segundo operando`
devolver $(x\ t_i\ y,\ k)$ `// aplicar el operador` t_i

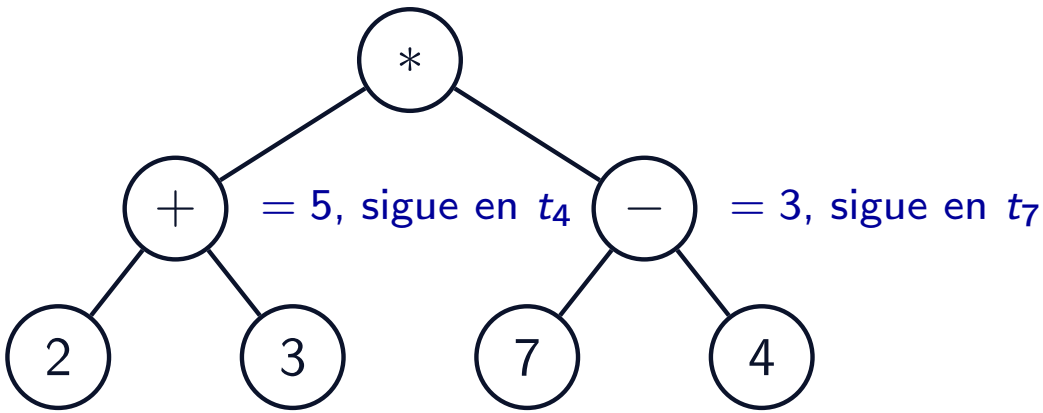
Es la definición recursiva escrita como algoritmo: el caso base es el número y el caso recursivo, el operador. Cada símbolo se lee una sola vez: $O(n)$ para n símbolos.

Evaluar $* \; + \; 2 \; 3 \; - \; 7 \; 4$



Cada nodo es una llamada; los símbolos son $t_0, \ldots, t_6$.

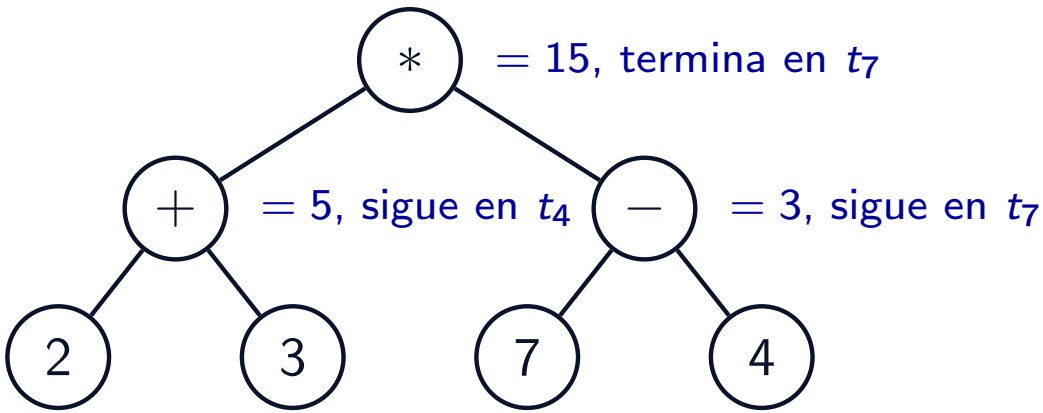
Evaluar $*\ +\ 2\ 3\ -\ 7\ 4$



Cada nodo es una llamada; los símbolos son $t_0, \ldots, t_6$.

EVALUAR(1) lee $+$, evalúa 2 y 3, y devuelve $(5, 4)$: el segundo operando de $*$ empieza en t_4.

Evaluar $*\ +\ 2\ 3\ -\ 7\ 4$



Cada nodo es una llamada; los símbolos son $t_0, \ldots, t_6$.

EVALUAR(1) lee $+$, evalúa 2 y 3, y devuelve $(5, 4)$: el segundo operando de $*$ empieza en t_4.

EVALUAR(4) devuelve $(3, 7)$, y EVALUAR(0) devuelve $(15, 7)$: se leyeron los 7 símbolos.

Paréntesis balanceados

Paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar todas las cadenas de n pares de paréntesis bien balanceados. Con $n = 3$: `((()))`, `(()())`, `(())()`, `()(())`, `()()()`.

Paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar todas las cadenas de n pares de paréntesis bien balanceados. Con $n = 3$: `((()))`, `(()())`, `(())()`, `()(())`, `()()()`.

Generar las 2^{2n} cadenas y filtrar las balanceadas funciona. ¿Se puede evitar construir las que no sirven?

Paréntesis balanceados

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar todas las cadenas de n pares de paréntesis bien balanceados. Con $n = 3$: `((()))`, `(()())`, `(())()`, `()(())`, `()()()`.

Generar las 2^{2n} cadenas y filtrar las balanceadas funciona. ¿Se puede evitar construir las que no sirven?

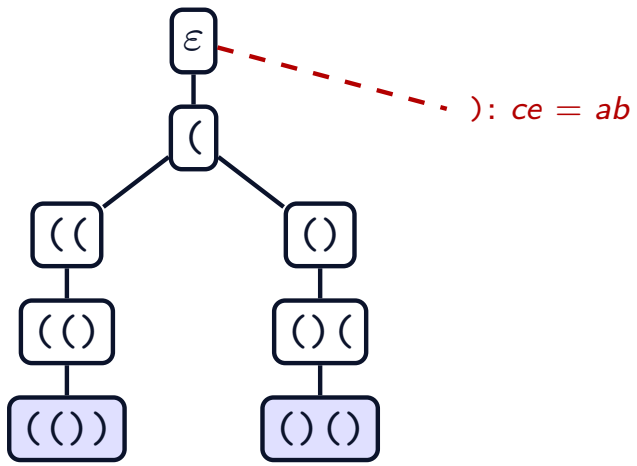
Sí: basta llevar dos contadores

Con ab paréntesis abiertos y ce cerrados: se puede agregar `(` si $ab < n$, y `)` si $ce < ab$. Una cadena armada con esas dos reglas nunca se desbalancea, así que toda hoja del árbol es una respuesta.

Paréntesis: pseudocódigo y árbol

Algoritmo 6: PARENTESIS(s, ab, ce)

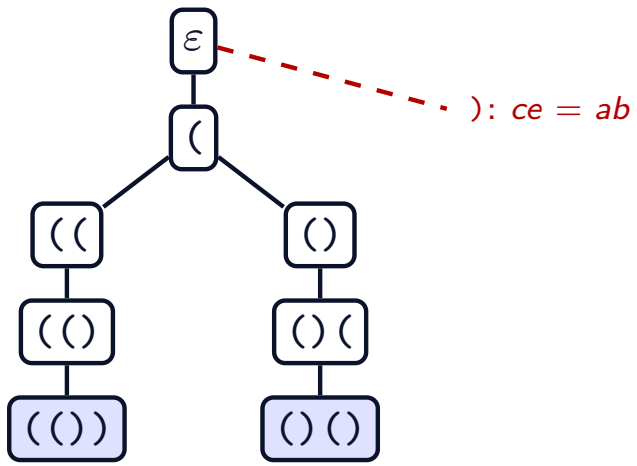
si $|s| = 2n$ **entonces**
  imprimir s
  devolver
si $ab < n$ **entonces**
  PARENTESIS$(s + (, ab + 1, ce)$
si $ce < ab$ **entonces**
  PARENTESIS$(s +), ab, ce + 1)$



Paréntesis: pseudocódigo y árbol

Algoritmo 7: PARENTESIS(s, ab, ce)

si $|s| = 2n$ **entonces**
  imprimir s
  devolver
si $ab < n$ **entonces**
  PARENTESIS$(s + (, ab + 1, ce)$
si $ce < ab$ **entonces**
  PARENTESIS$(s +), ab, ce + 1)$



El árbol para $n = 2$: la rama punteada nunca se crea. Para $n = 3$ hay 22 llamadas y 5 hojas, contra $2^6 = 64$ cadenas si se generara todo.

¿Cuántas cadenas hay?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

n	1	2	3	4	5	6	7	10
cadenas	1	2	5	14	42	132	429	16 796

¿Cuántas cadenas hay?

n	1	2	3	4	5	6	7	10
cadenas	1	2	5	14	42	132	429	16 796

Los números de Catalan

Crecen más o menos como 4^n: menos que $2^{2n} = 4^n$ por un factor polinomial, pero siguen siendo exponenciales. Como el algoritmo solo visita prefijos que terminan en una respuesta, su costo es proporcional a lo que imprime.

Permutaciones con letras repetidas

Permutaciones de una palabra

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar las palabras distintas que se forman reordenando las letras de `aabc`.

Permutaciones de una palabra

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar las palabras distintas que se forman reordenando las letras de aabc.

PERMUTACIONES de la clase anterior imprime 4! = 24 palabras.
¿Cuántas son distintas?

Permutaciones de una palabra

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Listar las palabras distintas que se forman reordenando las letras de `aabc`.

PERMUTACIONES de la clase anterior imprime $4! = 24$ palabras.
¿Cuántas son distintas?

$4!/2! = 12$: cada palabra sale dos veces

Las dos a son indistinguibles: intercambiarlas da la misma palabra. Filtrar repetidos al final obliga a generar todas y guardarlas; es mejor no generarlas.

Decidir qué letra, no qué posición

Algoritmo 8: PALABRAS(p) — $cnt[x]$: cuántas letras x quedan

```
si |p| = n entonces
    imprimir p
    devolver
para cada letra distinta x, en orden alfabético hacer
    si cnt[x] > 0 entonces
        cnt[x] ← cnt[x] − 1,  agregar x al final de p
        PALABRAS(p)
        quitar el último de p,  cnt[x] ← cnt[x] + 1
```

Decidir qué letra, no qué posición

Algoritmo 9: PALABRAS(p) — $cnt[x]$: cuántas letras x quedan

```
si |p| = n entonces
    imprimir p
    devolver
para cada letra distinta x, en orden alfabético hacer
    si cnt[x] > 0 entonces
        cnt[x] ← cnt[x] − 1,  agregar x al final de p
        PALABRAS(p)
        quitar el último de p,  cnt[x] ← cnt[x] + 1
```

En cada posición se prueba cada **letra distinta** una sola vez: dos ramas distintas del árbol siempre difieren en alguna posición, así que no hay repetidos. Salen en orden alfabético: `aabc`, `aacb`, `abac`, . . . , `cbaa`.

Sudoku

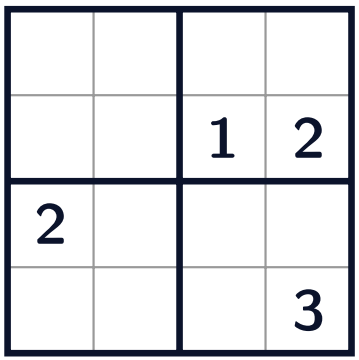
Sudoku de 4×4

		1	2
2			
			3

El problema

Completar el tablero de modo que cada fila, cada columna y cada bloque de 2×2 tenga los números 1 a 4.

Sudoku de 4×4



El problema

Completar el tablero de modo que cada fila, cada columna y cada bloque de 2×2 tenga los números 1 a 4.

El modelo de backtracking

Las casillas vacías se llenan en orden (fila por fila). En cada una se prueban los valores 1 a 4 que no choquen con su fila, su columna ni su bloque; si ninguno sirve, se retrocede.

Sudoku de 4×4

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

		1	2
2			
			3

El problema

Completar el tablero de modo que cada fila, cada columna y cada bloque de 2×2 tenga los números 1 a 4.

El modelo de backtracking

Las casillas vacías se llenan en orden (fila por fila). En cada una se prueban los valores 1 a 4 que no choquen con su fila, su columna ni su bloque; si ninguno sirve, se retrocede.

Tres tablas de booleanos, $fila[r][v]$, $columna[c][v]$ y $bloque[b][v]$, hacen cada verificación en $O(1)$.

Sudoku: pseudocódigo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 10: SUDOKU(k) — llena las casillas vacías $k, k+1, \ldots$

si *k pasó la última casilla vacía* **entonces devolver** VERDADERO
$(r, c) \leftarrow$ la k-ésima casilla vacía, $b \leftarrow$ su bloque
para $v \leftarrow 1$ **a** 4 **hacer**
  si ***no*** *fila*$[r][v]$ ***y no*** *columna*$[c][v]$ ***y no*** *bloque*$[b][v]$ **entonces**
    poner v en (r, c) y marcar las tres tablas
    si SUDOKU($k+1$) **entonces devolver** VERDADERO
    quitar v de (r, c) y desmarcar las tres tablas
devolver FALSO

Sudoku: pseudocódigo

Algoritmo 11: SUDOKU(k) — llena las casillas vacías $k, k+1, \ldots$

```
si k pasó la última casilla vacía entonces devolver VERDADERO
(r, c) ← la k-ésima casilla vacía,  b ← su bloque
para v ← 1 a 4 hacer
    si no fila[r][v] y no columna[c][v] y no bloque[b][v] entonces
        poner v en (r, c) y marcar las tres tablas
        si SUDOKU(k + 1) entonces devolver VERDADERO
        quitar v de (r, c) y desmarcar las tres tablas
devolver FALSO
```

Devuelve VERDADERO apenas completa el tablero, así que se detiene en la primera solución en vez de buscarlas todas.

Sudoku: un retroceso

1	2	3	4
3	4	1	2
2	1	4	?
			3

fila 2: falta el 3, pero ya está en su columna

		1	2
2			
			3

		1	2
2			
			3

Con el 1 en la casilla $(2, 1)$, la fila 2 queda $2, 1, 4,$?: el único valor posible es el 3, que choca con el 3 de la última fila.

Sudoku: un retroceso

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1	2	3	4
3	4	1	2
2	1	4	?
			3

fila 2: falta el 3, pero ya está en su columna

1	2	3	4
3	4	1	2
2	3	4	1
			3

se retrocede: 3 en $(2, 1)$

		1	2
2			
			3

Con el 1 en la casilla $(2, 1)$, la fila 2 queda $2, 1, 4,$?: el único valor posible es el 3, que choca con el 3 de la última fila.

Se vuelve a $(2, 2)$, que no tiene otro valor, y luego a $(2, 1)$: el siguiente valor que no choca es el 3, y ahora la fila 2 se completa con 4 y 1.

Sudoku: un retroceso

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

1	2	3	4
3	4	1	2
2	1	4	?
			3

fila 2: falta el 3, pero ya está en su columna

1	2	3	4
3	4	1	2
2	3	4	1
			3

se retrocede: 3 en $(2, 1)$

1	2	3	4
3	4	1	2
2	3	4	1
4	1	2	3

la última fila sale sin problemas

Con el 1 en la casilla $(2, 1)$, la fila 2 queda $2, 1, 4,$?: el único valor posible es el 3, que choca con el 3 de la última fila.

Se vuelve a $(2, 2)$, que no tiene otro valor, y luego a $(2, 1)$: el siguiente valor que no choca es el 3, y ahora la fila 2 se completa con 4 y 1.

¿Y el Sudoku de 9×9?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El mismo algoritmo

Con $v \leftarrow 1 \ldots 9$ y bloques de 3×3. El peor caso sigue siendo exponencial, pero las verificaciones cortan casi todas las ramas apenas empiezan.

¿Y el Sudoku de 9×9?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El mismo algoritmo

Con $v \leftarrow 1 \ldots 9$ y bloques de 3×3. El peor caso sigue siendo exponencial, pero las verificaciones cortan casi todas las ramas apenas empiezan.

¿En qué orden conviene llenar las casillas?

¿Y el Sudoku de 9×9?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El mismo algoritmo

Con $v \leftarrow 1 \ldots 9$ y bloques de 3×3. El peor caso sigue siendo exponencial, pero las verificaciones cortan casi todas las ramas apenas empiezan.

¿En qué orden conviene llenar las casillas?

Primero la que tiene menos opciones

Si una casilla admite un solo valor, ponerlo primero no genera ramas; si admite cero, se retrocede de inmediato. Elegir en cada paso la casilla con menos valores posibles poda muchísimo más que el orden fila por fila.

Para practicar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problemas clásicos de backtracking (UVa)

- *8 Queens Chess Problem* (UVa 750): todas las soluciones de las 8 reinas con una reina en una casilla dada.
- *Back to the 8-Queens* (UVa 11085): la menor cantidad de movimientos para convertir un tablero en una solución.

Para practicar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problemas clásicos de backtracking (UVa)

- *8 Queens Chess Problem* (UVa 750): todas las soluciones de las 8 reinas con una reina en una casilla dada.
- *Back to the 8-Queens* (UVa 11085): la menor cantidad de movimientos para convertir un tablero en una solución.
- *The Hamming Distance Problem* (UVa 729): todas las cadenas binarias de largo N con exactamente H unos, en orden.
- *Marcus* (UVa 10452): recorrer una grilla pisando solo las letras de una palabra, en orden.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La receta: predicado (en palabras), casos base y paso recursivo, confiando en el predicado. Si el paso no sale, se fortalece el predicado, como en Hanói.
- Hanói: $T(n) = 2T(n-1) + 1 = 2^n - 1$ movimientos, y no se puede con menos. Notación prefija: la recursión devuelve el valor y dónde terminó.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La receta: predicado (en palabras), casos base y paso recursivo, confiando en el predicado. Si el paso no sale, se fortalece el predicado, como en Hanói.
- Hanói: $T(n) = 2T(n-1) + 1 = 2^n - 1$ movimientos, y no se puede con menos. Notación prefija: la recursión devuelve el valor y dónde terminó.
- Buenas reglas de poda evitan construir lo que no sirve: paréntesis con dos contadores, palabras sin repetidos contando letras.
- Sudoku: backtracking casilla por casilla que se detiene en la primera solución; conviene llenar primero la casilla más restringida.

Logros de la sesión

Logros de la sesión

- Sabemos plantear una solución recursiva a partir de una instancia más chica, y contar su costo con la misma recursión.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos plantear una solución recursiva a partir de una instancia más chica, y contar su costo con la misma recursión.
- Sabemos escribir recursiones que devuelven más de un dato, como el valor y la posición.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos plantear una solución recursiva a partir de una instancia más chica, y contar su costo con la misma recursión.
- Sabemos escribir recursiones que devuelven más de un dato, como el valor y la posición.
- Sabemos diseñar reglas de poda para que el backtracking no genere candidatos inválidos ni repetidos.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias